

****分布式系统中的一致性模型****

马浩琨

Oct 09, 2026

在分布式系统中，状态被复制到多个节点上，而网络本身并不提供可靠的交付保证。一次写入可能只到达部分副本，读请求也可能落在不同的副本上，这就意味着同一逻辑对象在不同节点上可能出现不同版本。如果我们希望上层应用不必关心这种差异，就必须在系统层面建立“一致性”契约，让分布式存储在语义上等价于一台单机。一致性、可用性与分区容错性并非可以同时最大化。CAP 定理指出，当网络发生分区时，系统只能在一致性与可用性之间二选一。这里的“一致性”特指线性一致性，而更宽松的模型（如最终一致性）则可以与可用性共存。理解不同模型的精确含义，有助于工程师在设计或选型时做出权衡。本文面向已掌握分布式系统基本概念，但尚未系统梳理过一致性谱系的读者。文章将从严格一致性出发，沿着“强→弱”方向逐一展开，帮助读者建立一条连续的认知曲线。

1 背景：分布式存储的抽象模型

为了对一致性模型进行形式化讨论，我们首先需要一套共同的语言。系统由若干进程组成，每个进程可对共享对象发起读或写操作。一次操作在客观时钟上具有开始和结束两个瞬时，但由于时钟不同步，我们更关注操作之间的偏序关系。

Lamport 在 1978 年提出逻辑时钟，用一个单调递增的标量值捕捉“发生于之前”的因果关系。若事件 a 在同一进程内先于 b ，或 a 是跨进程消息的发送而 b 是接收，则可记 $a \rightarrow b$ 。通过对这种关系取传递闭包，我们得到一个偏序集，称为“历史”。线性历史要求任意两个操作都可比较，而分支历史则允许并发的操作互不可比。后续所有模型的定义，都建立在这套偏序语言之上。

2 强一致性模型

2.1 严格一致性

严格一致性要求：若写操作 w 在客观时刻 t 完成，则任何在 t 之后发起的读，都必须返回 w 写入的值或更新的值。这需要所有节点共享一个完美同步的全局时钟，并在写操作完成前让所有副本可见。现实中，石英晶振的漂移率约为 10^{-10} ，跨机房光速延迟也无法忽略，因此严格一致性几乎无法实现。它更多是理论上的参照点，用于衡量其他模型的差距。

2.2 顺序一致性

顺序一致性放宽了客观时钟的要求，但仍要求存在一个全局全序，使得：第一，所有进程看到的操作顺序一致；第二，该顺序与每个进程内部的执行顺序相容。换言之，只要进程内部的操作保持相对顺序，跨进程的操作可以

重排。1979 年 Lamport 在多处理器环境下首次提出这一概念，用于证明缓存一致性协议的正确性。

2.3 线性一致性

线性一致性在顺序一致性的基础上增加了实时性约束：若操作 a 在客观时钟上先于 b 开始，则 a 在全局顺序中必须排在 b 之前。Herlihy 与 Wing 在 1990 年给出了形式化定义，并证明线性一致性具有可组合性——多个线性一致对象组合后依然线性一致。原子寄存器、CAS 等硬件原语，都可视为线性一致对象。在事务语境中，线性一致性对应 ACID 中的“A”，即原子性可见性。

三者强度关系为：严格一致性强于线性一致性，线性一致性强于顺序一致性。严格一致性要求全局时钟，线性一致性仅要求操作的实时序，顺序一致性则只要求逻辑序。

3 因果一致性

因果一致性关注“happened-before”关系：若 $a \rightarrow b$ ，则所有进程都必须让 a 先于 b 被观察到；若 a 与 b 并发，则允许不同进程以不同顺序观察它们。实现这一模型最常用的工具是向量时钟。每个副本维护一个长度等于节点数的向量，向量的第 i 个分量记录该副本所知的节点 i 的最新事件编号。收到消息时，向量做分量-wise 取最大；发送消息时，本地分量自增后再附在消息上。通过比较向量，我们可以判断两个事件是否存在因果依赖，从而在冲突解决时保留偏序。

在社交网络场景中，用户 A 发布动态、用户 B 评论该动态，这两件事存在因果链。向量时钟可以保证任意第三方用户不会先看到评论而后看到动态，但 A、B 之外的其他并发动态则允许乱序到达。

4 弱一致性与最终一致性

最终一致性只做一条承诺：在更新停止后，若等待足够长的时间，所有副本将收敛到相同状态。它的工程价值在于允许分区期间继续接受写入，从而最大化可用性。Amazon 在 Dynamo 论文中提出 BASE 口号：基本可用、软状态、最终一致。软状态指系统在一段时间内可以接受中间状态，而不必立即达到一致。

工程上，Dynamo 式系统通常采用读写仲裁。以复制因子 $N=3$ 、 $W=2$ 、 $R=2$ 为例，写请求必须收到至少两个副本确认，读请求也必须联系两个副本。只要 $W+R>N$ ，就可保证读到最新值，否则可能读到旧值。向量时钟用于检测冲突版本，读修复在读路径上异步补齐落后副本，反熵则在后台定期全量比对。

一致性窗口用来度量收敛时间。Amazon 曾公开其购物车服务在正常情况下 99.9% 的请求可在 5 ms 内收敛。这是一个概率性 SLA，允许极少数请求经历更长尾延迟。

5 一致性模型与 CAP、PACELC

CAP 关注分区发生时的取舍：CP 系统在分区时拒绝写入以保护一致性，如 HBase；AP 系统在分区时继续写入，事后解决冲突，如 Cassandra。线性一致性天然对应 CP，因为它要求分区两侧的读必须能互相看到最新值。最终一致性则对应 AP。

PACELC 将 CAP 扩展到无分区场景：若发生分区，则在一致性与可用性中权衡；否则，在一致性与延迟中权衡。下表给出四种主流系统的 PACELC 配置：

MongoDB 默认使用多数派写、多数派读，属于 PC/EL；Cassandra 支持可调仲裁，属于 PA/EL；PNUTS 采用主从异步复制，属于 PA/PC；DynamoDB 在全球表模式下属于 PA/EL。

6 更高阶模型与混合策略

6.1 会话一致性

会话一致性在单个客户端会话内提供更强的保证。读己之写要求客户端写入后立即能读到自己的更新；单调读要求客户端不会看到时间倒流；单调写要求同一客户端的写入按其发起顺序生效；写后读则要求写操作完成前不会被后续读追上。Cassandra 通过在协调者节点上绑定会话令牌来实现这些保证。

6.2 限定污点

限定污点在最终一致性的框架下增加时间或版本边界。例如 Azure Cosmos DB 的有界失效模式允许用户声明“最多滞后 5 秒或 200 个版本”。系统通过在副本间周期性心跳来估算滞后，并在超过边界时拒绝读请求，从而把概率性承诺变成可量化的 SLA。

6.3 可调一致性

Cassandra 的 ONE、QUORUM、ALL 级别允许客户端在每次请求中指定 W 和 R。ONE 提供最低延迟但可能读到旧值，ALL 提供最高一致性但在节点故障时不可用。客户端可根据业务语义动态选择，例如支付流程使用 QUORUM，日志采集使用 ONE。

6.4 混合一致性

同一系统内部可以共存不同模型。Google Spanner 用 TrueTime API 给事务打上全球单调的时间戳，再结合 Paxos 实现线性一致的分布式事务；同时，其存储层对只读索引使用最终一致的 follower read，从而在全局强一致与局部低延迟之间取得平衡。

7 实际案例：从理论到选型

银行转账要求借贷同时生效或同时失败，属于线性一致性；库存扣减允许先扣后补，但必须保证不超卖，可用因果一致性；点赞计数则可接受短暂不一致，最终收敛即可。

Spanner 通过 TrueTime 硬件时钟把提交延迟控制在 10 ms 量级；TiDB 依赖 PD 的 TSO，延迟约为 50 ms；CockroachDB 使用混合逻辑时钟，延迟介于两者之间。三者都提供线性一致性，但对时钟的依赖方式不同。在一致性、延迟与吞吐之间画成本曲线时，可先用生产流量重放测出不同一致性级别下的 P99 延迟，再结合业务可接受的错误率，找到最优的 W/R 组合。

一致性模型是一条连续谱，而非离散选项。硬件时钟精度提升（如 TrueTime、PTP）正在降低强一致性的成本，但最终一致性仍将在高延迟、弱网络环境下占据重要位置。读者在选型时应首先回答：业务场景真正需要线性一致性吗？其次，如何度量并监控一致性窗口？只有把理论映射到可观测指标，才能在工程实践中持续优化权衡。

8 参考文献与延伸阅读

Lamport, L. (1979). How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs. IEEE Transactions on Computers.

Herlihy, M. P., & Wing, J. M. (1990). Linearizability: A Correctness Condition for Concurrent Objects. ACM Transactions on Programming Languages and Systems.

Brewer, E. A. (2000). Towards Robust Distributed Systems. PODC Keynote.

Abadi, D. J. (2012). Consistency Tradeoffs in Modern Distributed Database System Design. IEEE Computer.

Corbett, J. C., et al. (2012). Spanner: Google's Globally-Distributed Database. OSDI.
etcd, TiKV, Cassandra 源代码中一致性协议的实现。

9 附录

9.1 附录 A 线性一致性验证伪代码

```
1 def is_linearizable(history):  
2     """  
3     history: 列表，每个元素形如 (op, obj, val, start_ts, end_ts)  
4     返回布尔值，表示给定历史是否满足线性一致性。  
5     核心思想：寻找一个与实时序相容的全局顺序，使对象表现为原子。  
6     """  
7     # 1. 按结束时间排序，得到可能的线性扩展顺序  
8     sorted_ops = sorted(history, key=lambda x: x[4])  
9     # 2. 模拟单机执行，检查读返回值是否与最新写一致  
10    state = {}  
11    for op, obj, val, _, _ in sorted_ops:  
12        if op == 'write':  
13            state[obj] = val  
14        elif op == 'read':  
15            if state.get(obj) != val:  
16                return False  
17    return True
```

上述代码首先把所有操作按结束时间排序，得到一个候选全局顺序。随后模拟单机执行：写操作直接更新本地状态，读操作则检查返回值是否与最新状态一致。若任意一次读不匹配，说明该顺序无法满足线性一致性，需要回溯或宣告失败。

9.2 附录 B 向量时钟冲突检测示例

```
1 def happens_before(vc_a, vc_b):  
    """  
3     vc_a, vc_b: 长度相同的向量, 元素为整数  
    若 vc_a 逐分量 ≤ vc_b 且至少一个分量 <, 则 a 因果先于 b  
5     """  
    le = all(x <= y for x, y in zip(vc_a, vc_b))  
7    lt = any(x < y for x, y in zip(vc_a, vc_b))  
    return le and lt  
9  
def detect_conflict(vc_a, vc_b):  
11    """  
    若 a 与 b 互不 happens_before, 则存在冲突  
13    """  
    return not happens_before(vc_a, vc_b) and not happens_before(vc_b, vc_a)
```

函数 `happens_before` 通过逐分量比较判断偏序关系。`detect_conflict` 利用其否定形式：若 `a` 既不先于 `b`，`b` 也不先于 `a`，则两事件并发，需要上层策略（如最后写入者获胜或应用层合并）来解决。

9.3 附录 C 各模型在时间线图上的可视化对比

严格一致性要求读操作必须落在最近一次写之后；线性一致性允许读在写结束后立即返回，但写与读之间不能重叠；顺序一致性允许写与读在客观时间上重叠，只要逻辑顺序与程序顺序相容；最终一致性则完全不约束顺序，只要求收敛。